

AWS API Gateway Lab Part 2: Create HTTP API and Perform Testing

To Begin with the Lab

Summary of the Lab

In this part of the lab, an **HTTP API** was created in **Amazon API Gateway** to expose the backend **AWS Lambda** functions as public API routes. A new API named UserAPI was set up, and routes were mapped so that **POST /user** triggered `createUserFunction`, **GET /user/{userId}** triggered `getUserFunction`, and **DELETE /user/{userId}** triggered `deleteUserFunction`. This made **API Gateway** the bridge between the frontend or Postman and the Lambda backend. The lab then tested the API end to end by sending create, read, and delete requests and confirming the expected changes in **Amazon DynamoDB**. It also confirmed the missing-record case by returning User not found after deletion.

Understanding API Gateway in This Lab

API Gateway provides a secure HTTP entry point for clients so they do not call **AWS Lambda** directly. In this lab, it receives request methods like **POST**, **GET**, and **DELETE**, then forwards each one to the correct Lambda function. This keeps the application organized and makes the backend easier to scale and maintain. Using an **HTTP API** also gives a lighter and faster setup for Lambda-based routes compared with a full REST API.

Lab Steps

Step 1 – Open API Gateway

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Click **Create API**.

API Gateway

APIs

Custom domain names
Domain name access associations
VPC links
AgentCore targets

Usage plans
API keys
Client certificates
Settings

Developer portals

Portals
Portal products

APIs (14/14)

Find APIs

	Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
☐	api-intelcore1	for testing	api-intelcore1	REST	Regional	2026-05-16	TLS_1_0	Available
☐	APIControl1	for demo	api-control1	REST	Regional	2026-01-07	TLS_1_0	Available
☐	APIControl2	for demo	api-control2	REST	Regional	2026-01-07	TLS_1_0	Available
☐	apigateway2-API	Created by AWS Lambda	api-gateway2	REST	Regional	2025-02-19	TLS_1_0	Available
☐	apigateway3		api-gateway3	REST	Regional	2025-02-19	TLS_1_0	Available
☐	apigateway4		api-gateway4	REST	Regional	2025-02-19	TLS_1_0	Available
☐	awsapigateway1-API	Created by AWS Lambda	api-gateway1	REST	Regional	2025-02-19	TLS_1_0	Available
☐	awslambdaap1-API	Created by AWS Lambda	api-lambda1	REST	Regional	2026-05-16	TLS_1_0	Available
☐	demoapi1	demoapi	api-demo1	REST	Regional	2025-04-28	TLS_1_0	Available
☐	IntelCoreWebAPI2	for demo	api-intelcore2	REST	Regional	2026-05-16	TLS_1_0	Available
☐	mydemo1		api-demo2	REST	Regional	2025-04-28	TLS_1_0	Available

Created by AWS

- Choose **HTTP API**.
- Click **Build**.

Choose an API type

HTTP API

Build low-latency and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

Works with the following:
Lambda, HTTP backends

Import Build

WebSocket API

Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.

Works with the following:
Lambda, HTTP, AWS Services

Build

REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Step 2 – Create the HTTP API

- Enter the API name: UserAPI
- Keep the default settings.

Step 1
● **Configure API**
● Step 2 - optional
Configure routes
● Step 3 - optional
Define stages
● Step 4
Review and create

Configure API

API details

API name
An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

☒ **IPv4**
Includes only IPv4 addresses.

☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

Integrations (0) [Info](#)
Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

[Add integration](#)

[Cancel](#) [Review and create](#) [Next](#)

- Leave routes and stages unchanged for now.
- Click Next.

Step 1
● Configure API
● Step 2 - optional
Configure routes
● **Step 3 - optional
Define stages**
● Step 4
Review and create

Define stages - optional

Configure stages [Info](#)

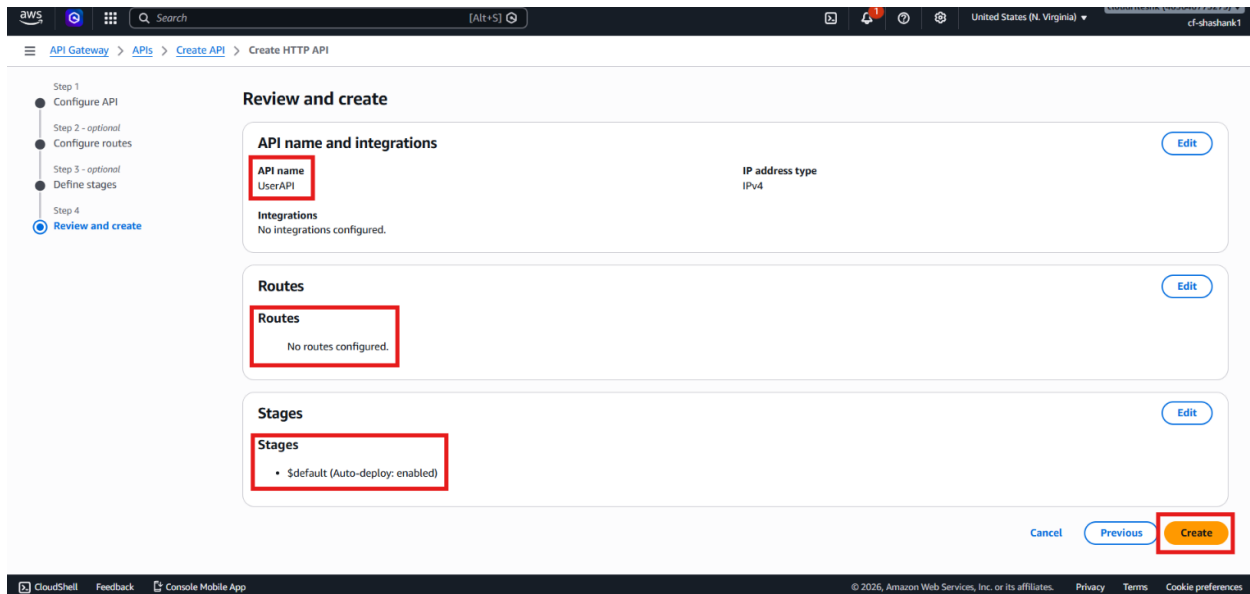
Stages are independently configurable environments that your API can be deployed to. You must deploy to a stage for API configuration changes to take effect, unless that stage is configured to autodeploy. By default, all HTTP APIs created through the console have a default stage named \$default. All changes that you make to your API are autodeployed to that stage. You can add stages that represent environments such as development or production.

Stage name **Auto-deploy** ☒ [Remove](#)

[Add stage](#)

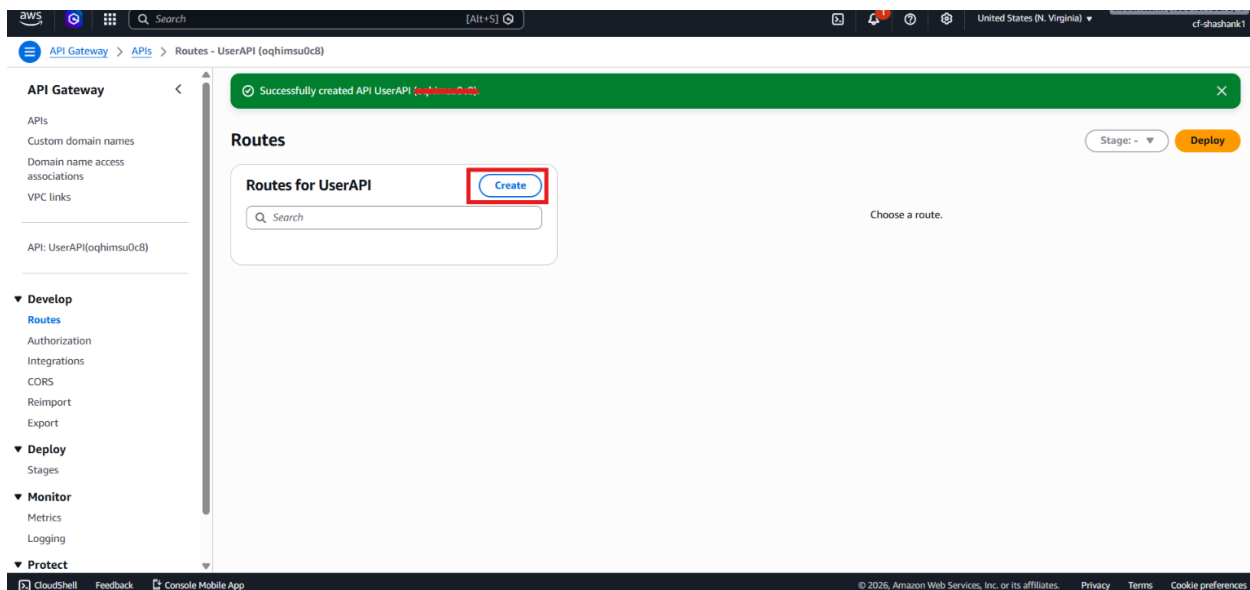
[Cancel](#) [Previous](#) [Next](#)

- Click Create.

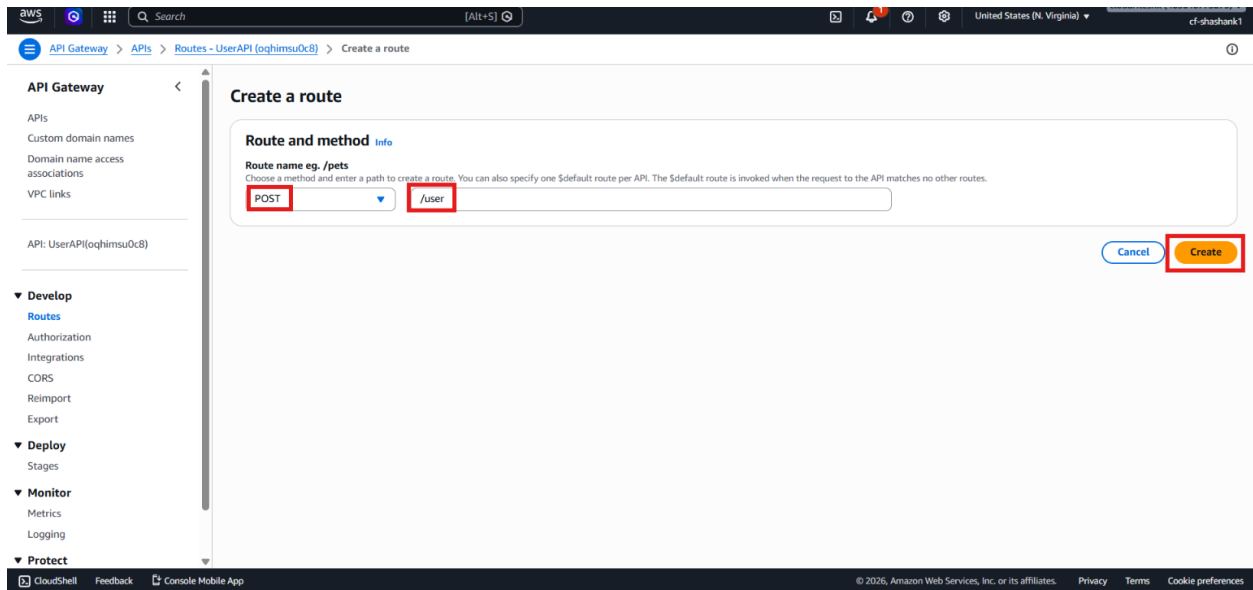


Step 3 – Create the Create User Route

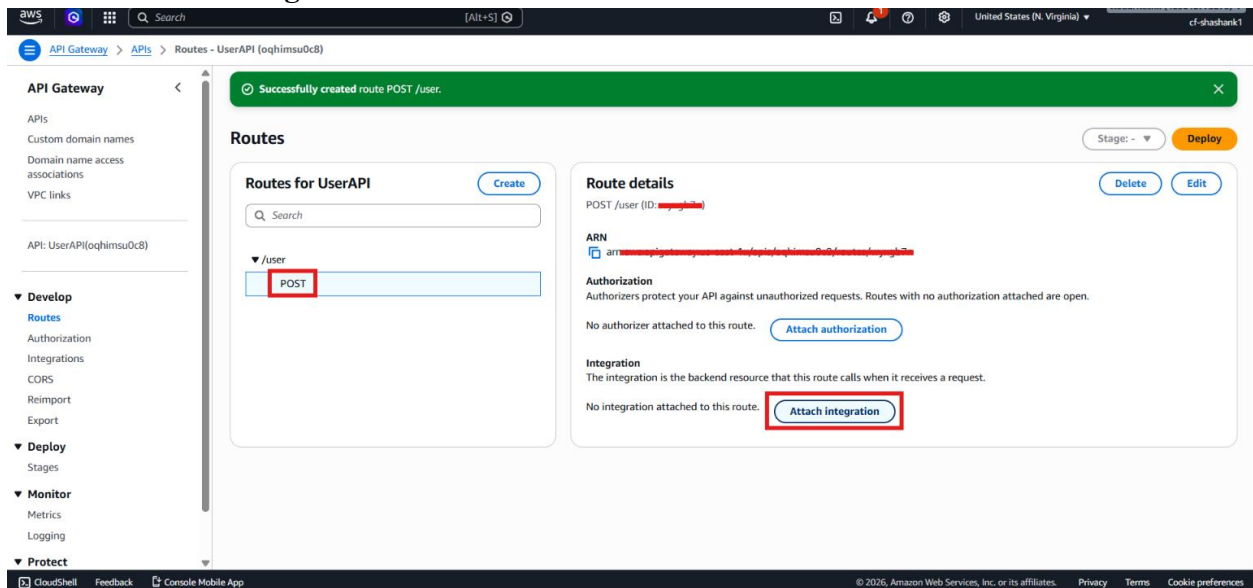
- Open UserAPI.
- Go to **Routes**.
- Click **Create**.



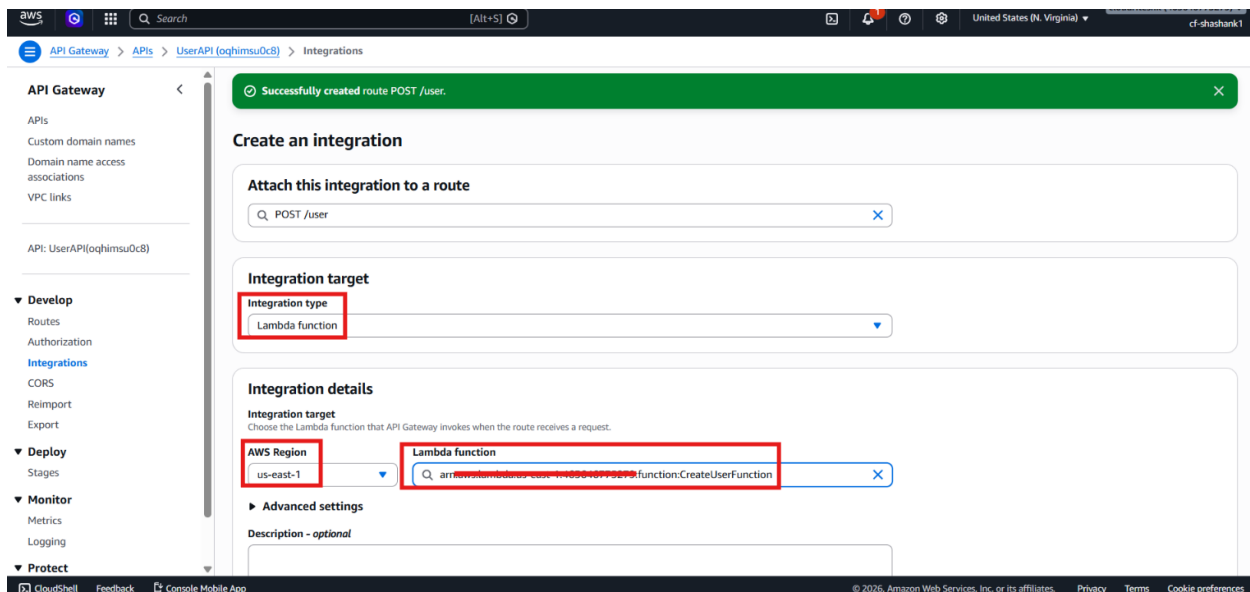
- Select **POST** as the method.
- Enter the route path: `/user`
- Click **Create**.



- Click **Attach integration**.



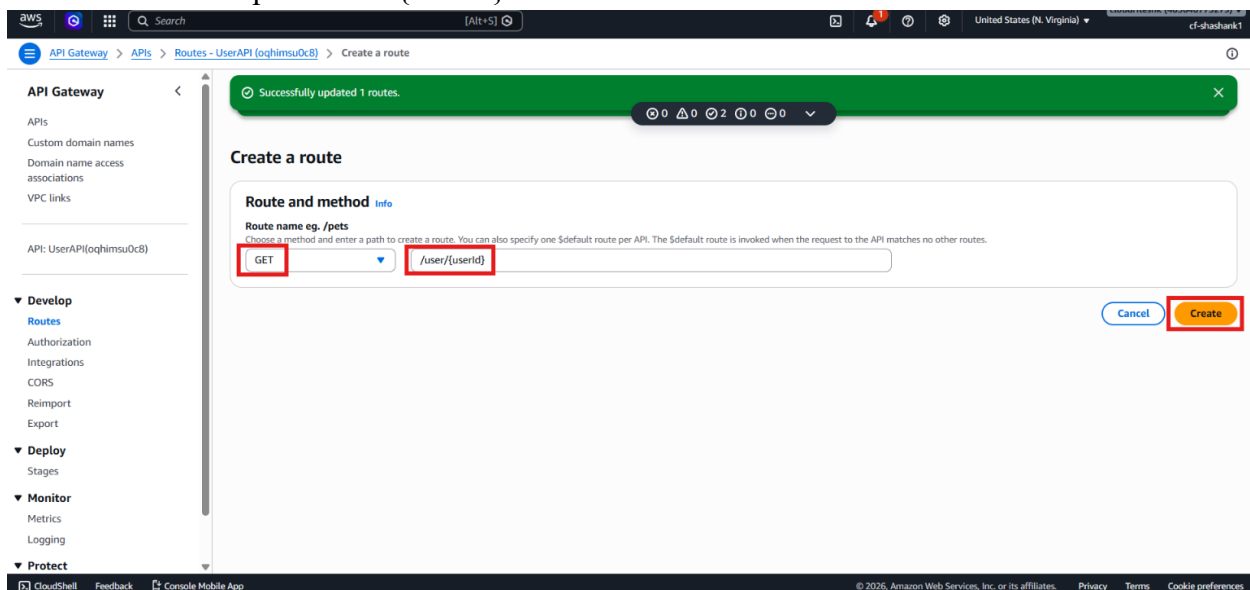
- Choose **Lambda function**.
- Select **createUserFunction**.



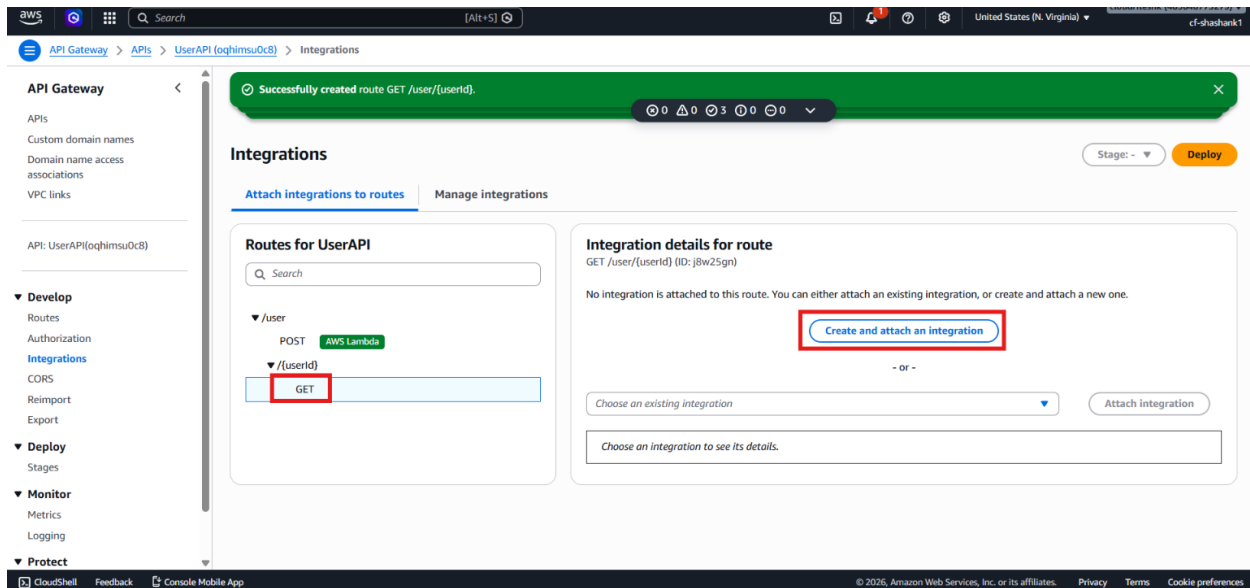
- Click **Create**.

Step 4 – Create the Get User Route

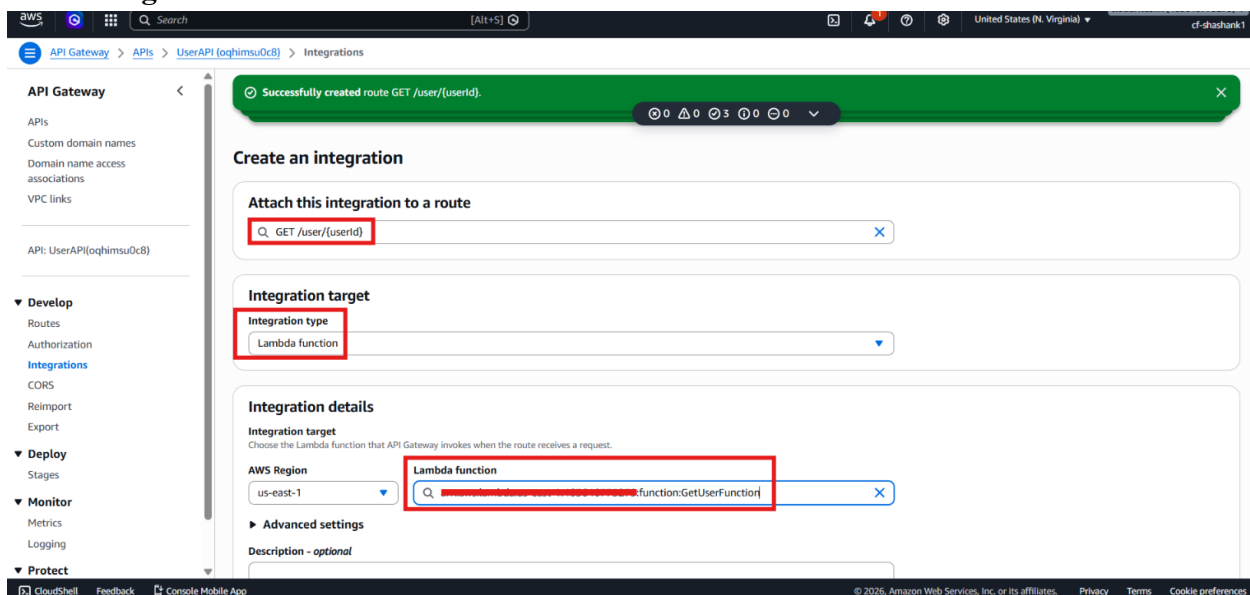
- Go back to **Routes**.
- Click **Create**.
- Select **GET** as the method.
- Enter the route path: `/user/{userId}`



- Click **Create**.
- Click **Attach integration**.



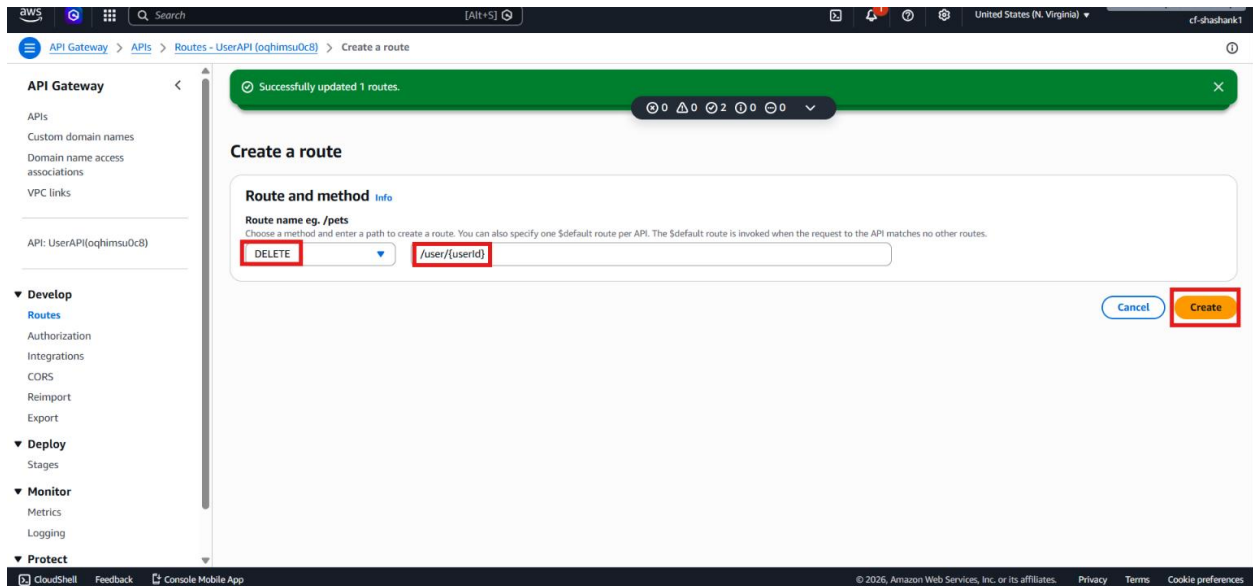
- Choose **Lambda function**.
- Select **getUserFunction**.



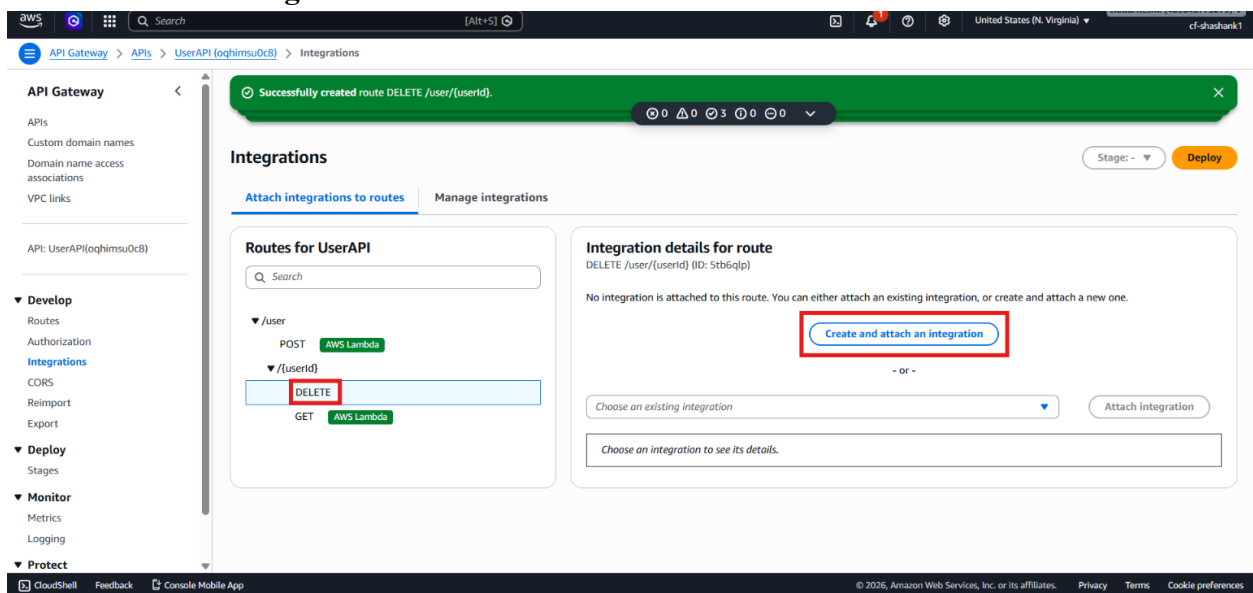
- Click **Create**.

Step 5 – Create the Delete User Route

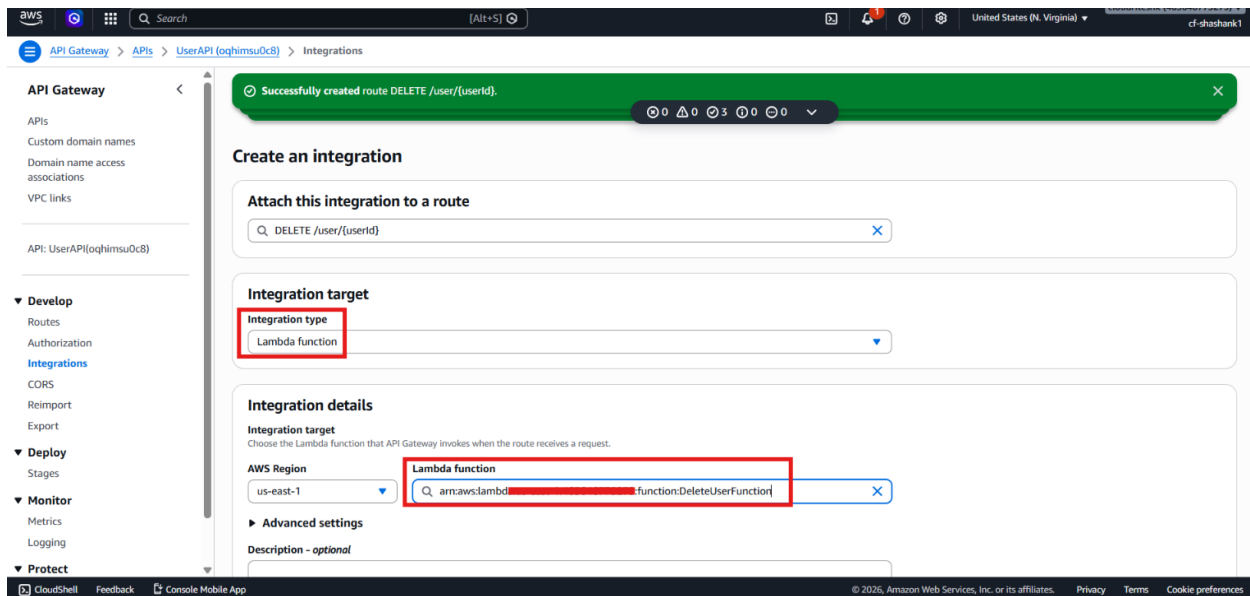
- Go back to **Routes**.
- Click **Create**.
- Select **DELETE** as the method.
- Enter the route path: `/user/{userId}`



- Click **Create**.
- Click **Attach integration**.



- Choose **Lambda function**.
- Select **deleteUserFunction**.



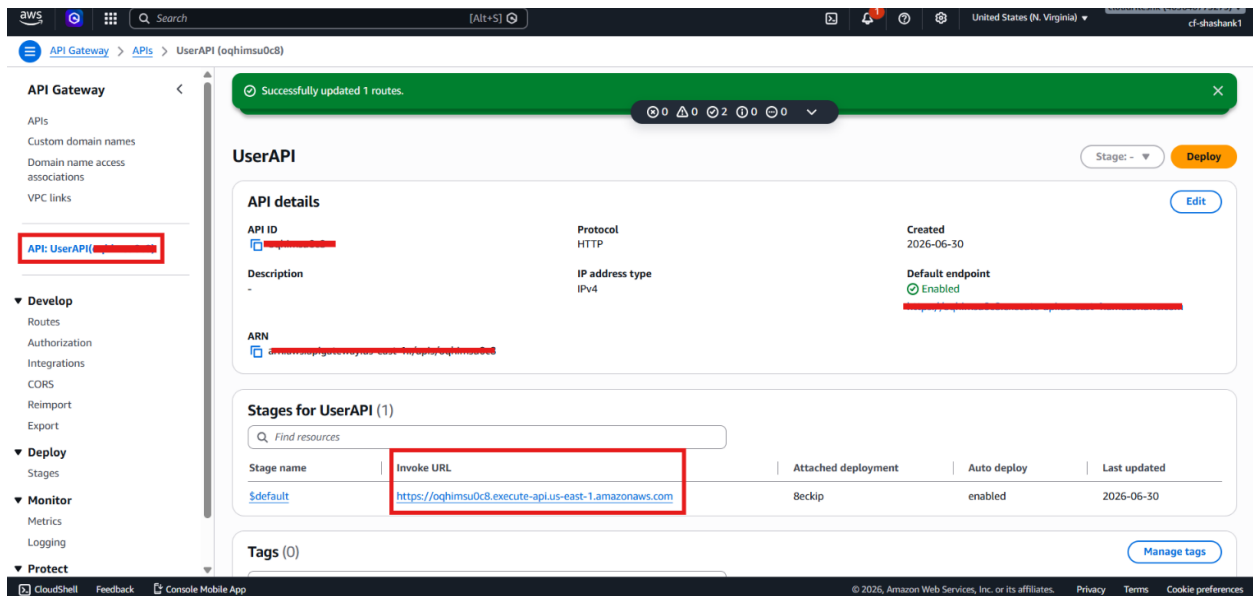
- Click **Create**.

Step 6 – Review the API Mapping

- Confirm the route mappings:
 - POST /user → **createUserFunction**
 - GET /user/{userId} → **getUserFunction**
 - DELETE /user/{userId} → **deleteUserFunction**
- Review the API trigger URL.
- Note that this is the URL the frontend or Postman will use to invoke the API.

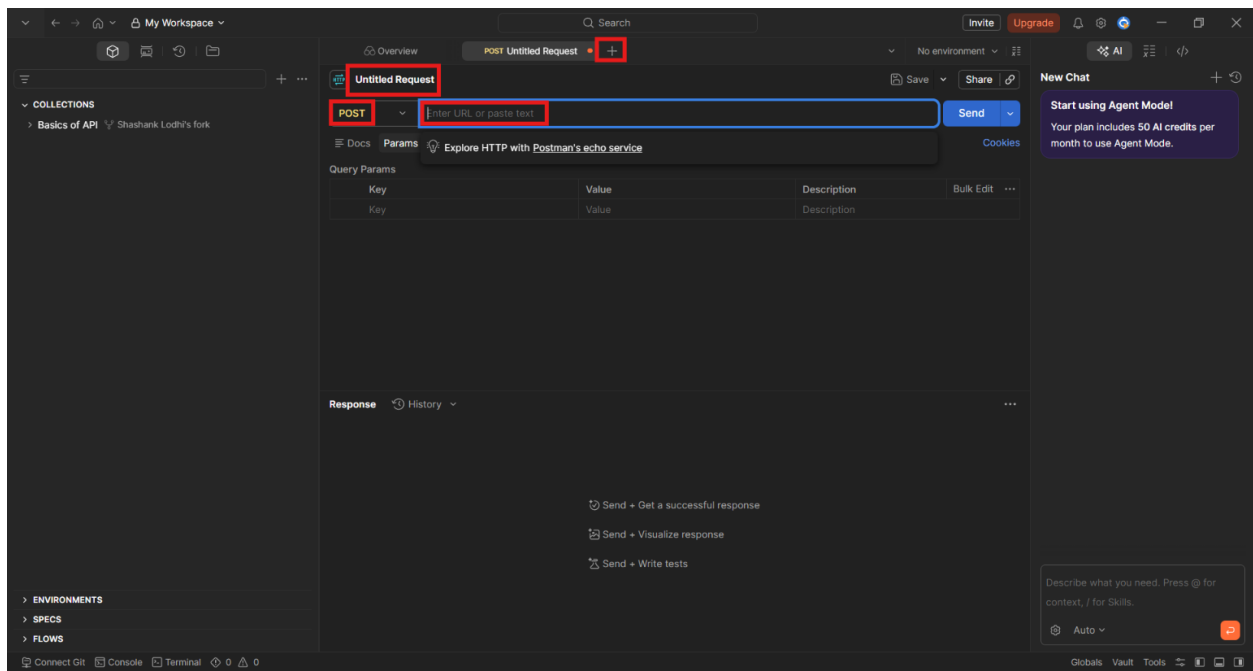
Step 7 – Open the API Gateway Invoke URL

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Select **UserAPI**.
- Go to **Stages**.
- Copy the **Invoke URL**.

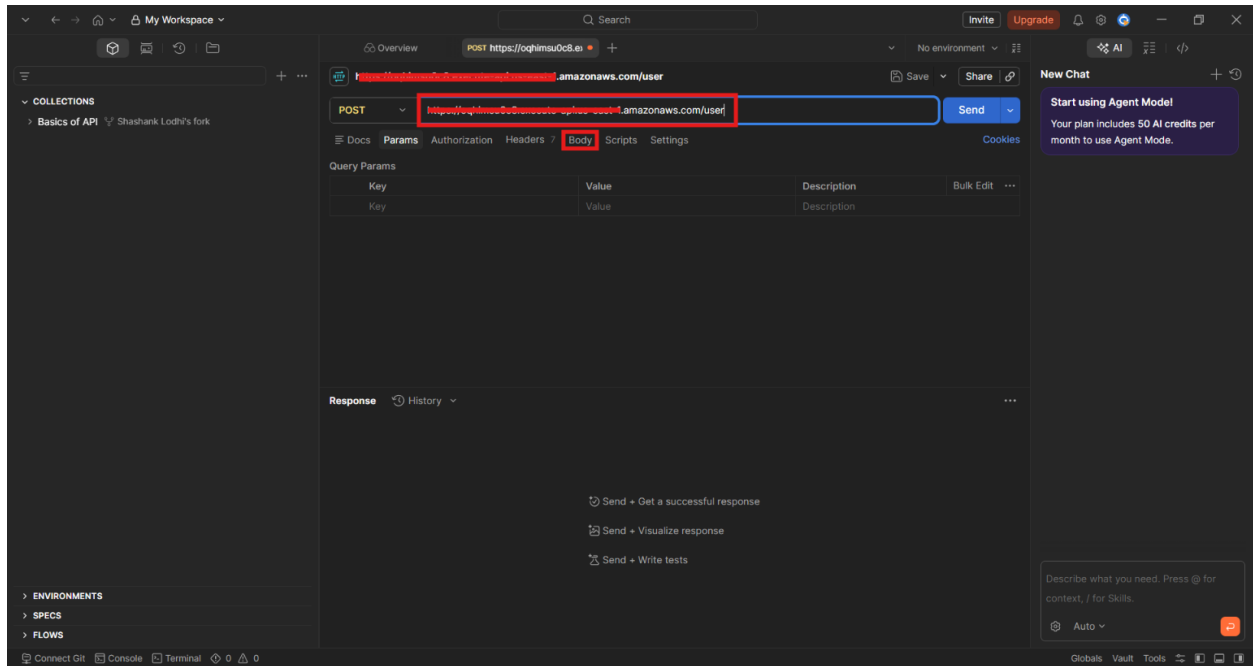


Step 8 – Test the Create User Request in Postman

- Open **Postman**.
- Create a new request.
- Set the method to **POST**.



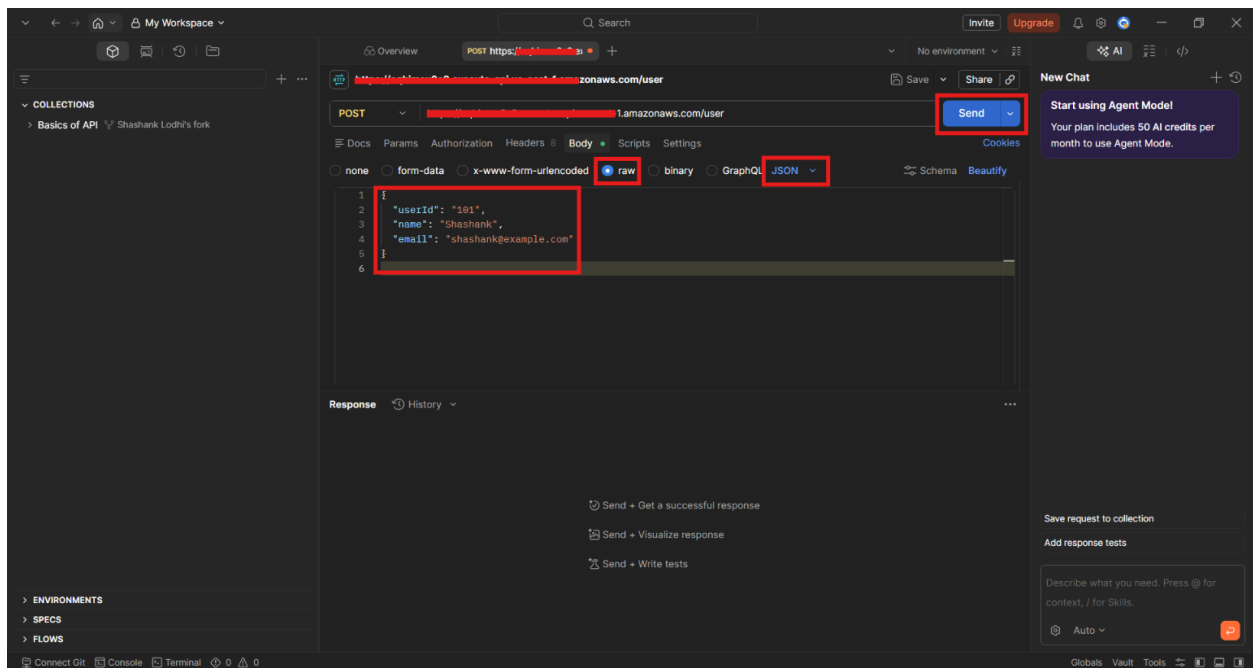
- Paste the invoke URL and append **/user**.
- Open the **Body** tab.



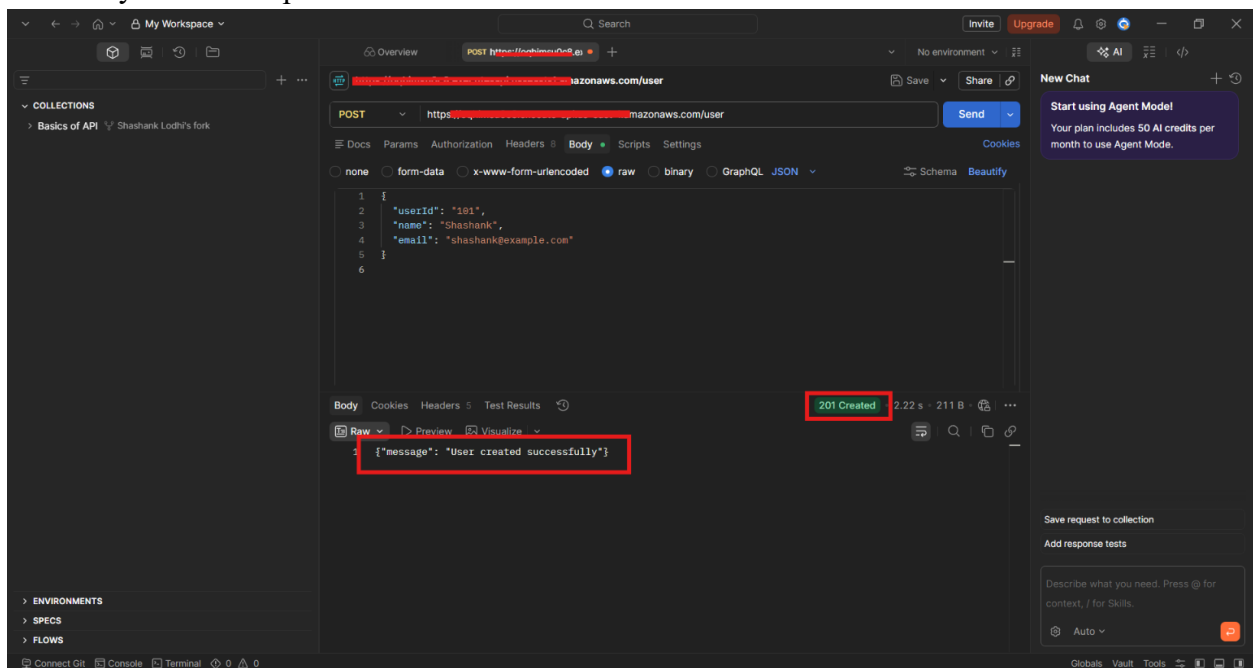
- Select **raw**.
- Choose **JSON** as the format.
- Add the user payload.

```
{  
  "userId": "101",  
  "name": "Shashank",  
  "email": "shashank@example.com"  
}
```

- Click **Send**.



- Verify that the response shows user creation success.



- Open Amazon DynamoDB.
- Go to **Tables** → **UsersTable**.

The screenshot shows the AWS Management Console for a DynamoDB instance. On the left, the 'DynamoDB' sidebar is visible. The main area shows the 'UsersTable' with a red box highlighting the 'Explore table items' button. The table's general information is displayed, showing it is active and has 0 items. The 'Partition key' is 'userid (String)' and the 'Sort key' is '-'. The 'Capacity mode' is 'On-demand'.

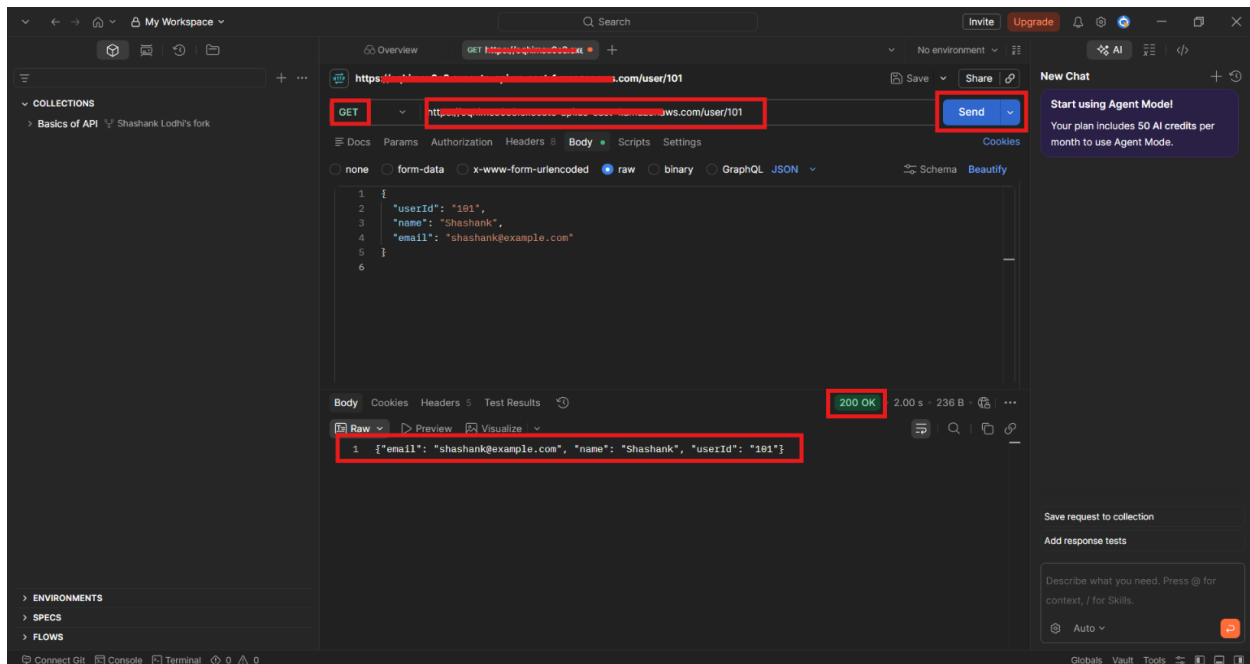
- Click **Explore table items**.
- Confirm that the record was inserted.

The screenshot shows the AWS Management Console for the 'UsersTable' with the 'Explore items' button selected. The 'Scan or query items' section is active, showing a completed scan operation. The scan results are displayed in a table with the following data:

userid (String)	email	name
101	shashank@...	Shashank

Step 9 – Test the Get User Request

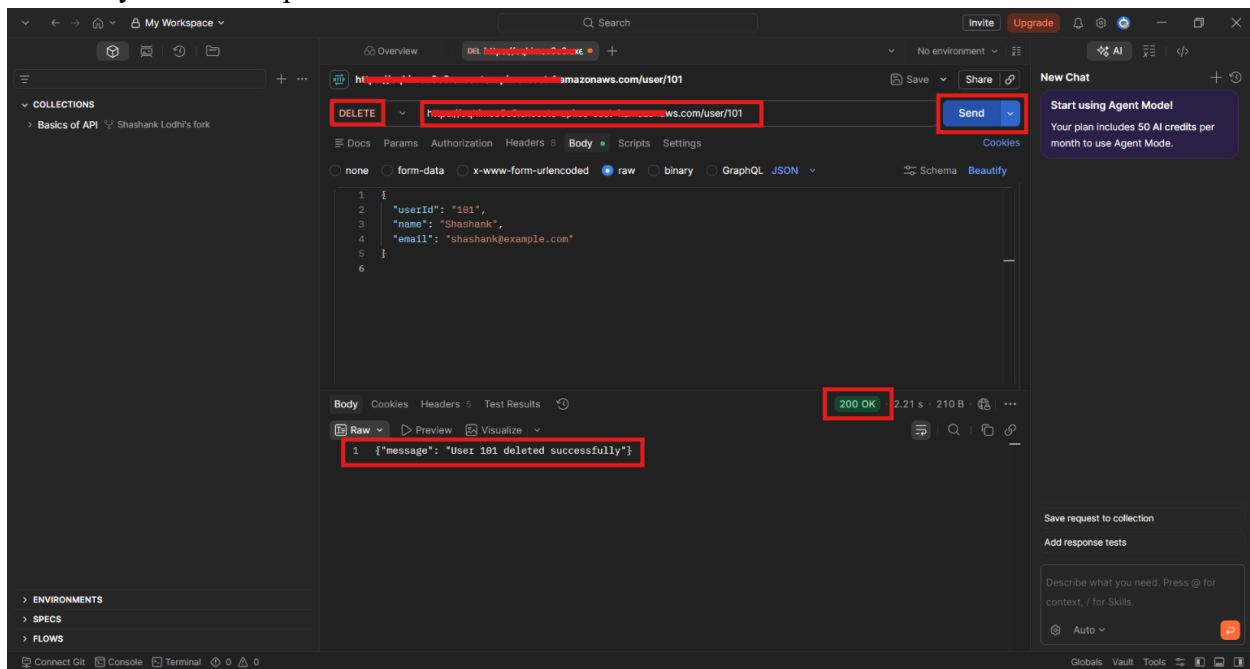
- In **Postman**, change the method to **GET**.
- Use the route **/user/101**.
- Click **Send**.
- Verify that the user details are returned.



- Confirm that the data matches the record stored in **UsersTable**.

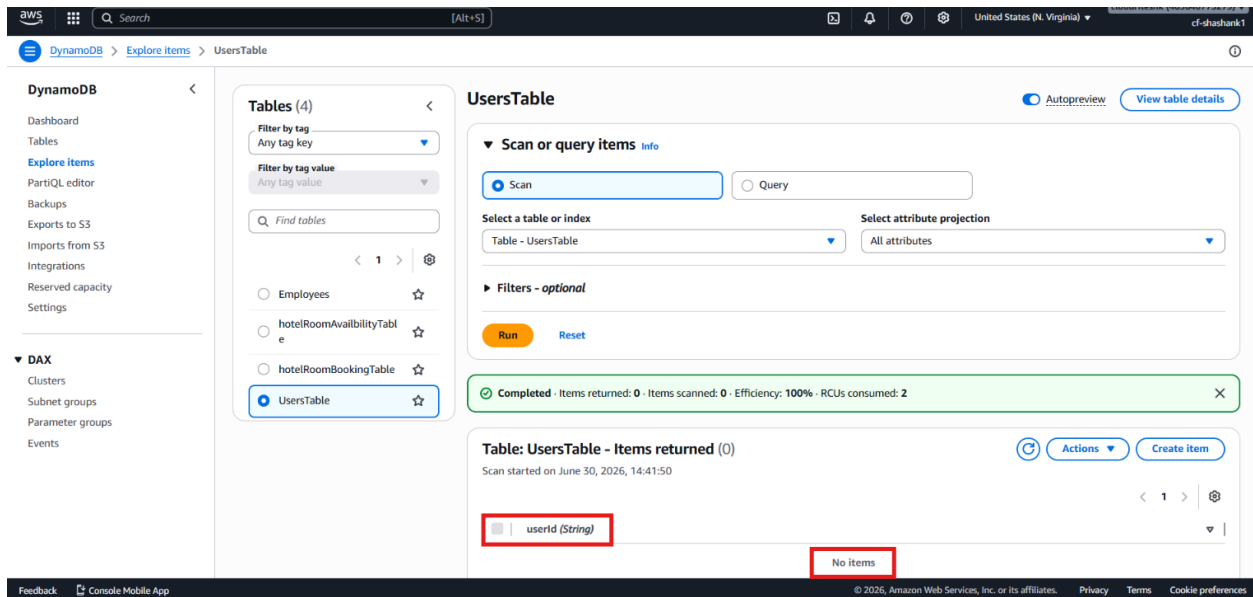
Step 10 – Test the Delete User Request

- In **Postman**, change the method to **DELETE**.
- Use the route **/user/101**.
- Click **Send**.
- Verify that the response confirms deletion.



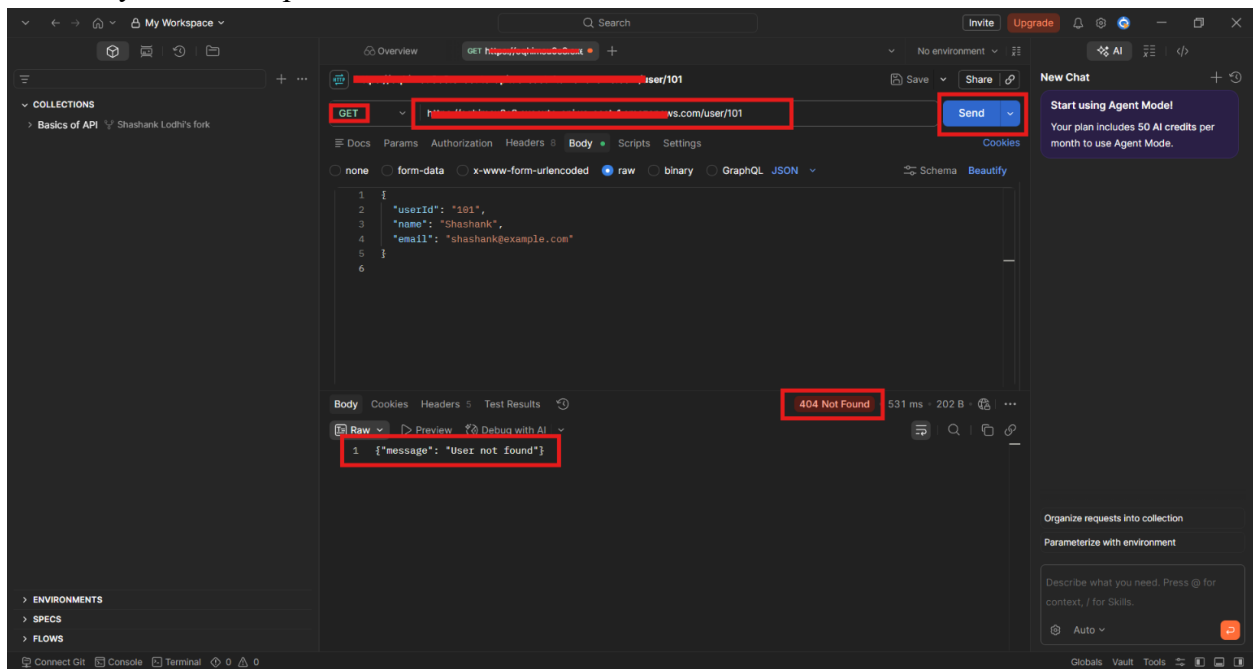
- Return to **Amazon DynamoDB**.
- Refresh **UsersTable**.

- Confirm that the record was removed.



Step 11 – Confirm the Error Case

- Send the same GET /user/101 request again after deletion.
- Verify that the response shows **User not found**.



- Confirm that the API correctly handles missing records.

Step 12 – Clean Up the API Gateway Resource

- Return to API Gateway.
- Select the **HTTP API** if you want to remove it.
- Delete the API Gateway resource only if the lab is complete.

- Keep the **Lambda** functions and **DynamoDB** table if you want to reuse them later.

Verification

- Verified that the **HTTP API** was created successfully in **Amazon API Gateway**.
- Verified that the route mappings were defined correctly for create, read, and delete operations.
- Verified that POST /user was connected to createUserFunction.
- Verified that GET /user/{userId} was connected to getUserFunction.
- Verified that DELETE /user/{userId} was connected to deleteUserFunction.
- Verified that the invoke URL was copied from the **Stages** section and used for testing.
- Verified that the POST request created a user record successfully in **Amazon DynamoDB**.
- Verified that the GET request returned the correct user details.
- Verified that the DELETE request removed the record from **Amazon DynamoDB**.
- Verified that a repeated GET /user/101 request after deletion returned User not found.
- Verified that the API was working as the bridge between the frontend and the Lambda backend.